System Architecture and Design Documentation

Contents

Document Control 2
1. Scope and Design Drivers 2
2. System Context 2
3. Logical Architecture 3
4. Deployment Architecture 4
5. Data Design 5
6. Processing Designs 6
7. Trust Boundaries and Security Responsibilities 7
8. Deployment, Upgrade and Rollback 7
9. Capacity, Observability and Recovery 8
10. Design Decisions and Verification 8

Document Control

Attribute	Value
Document ID	CRM-SDD-001
Product / publisher	AAP Fleet Management System / Selfmade Cloud Solutions
Version / issue date	1.0 / 13 November 2026
Source baseline	`5974cf7addfef5d5cfc8fde325fc964343557a2f`
Status	Technical review draft; actual production topology requires confirmation
Approvers	Commission architect, infrastructure owner and authorized signatory: TO BE CONFIRMED

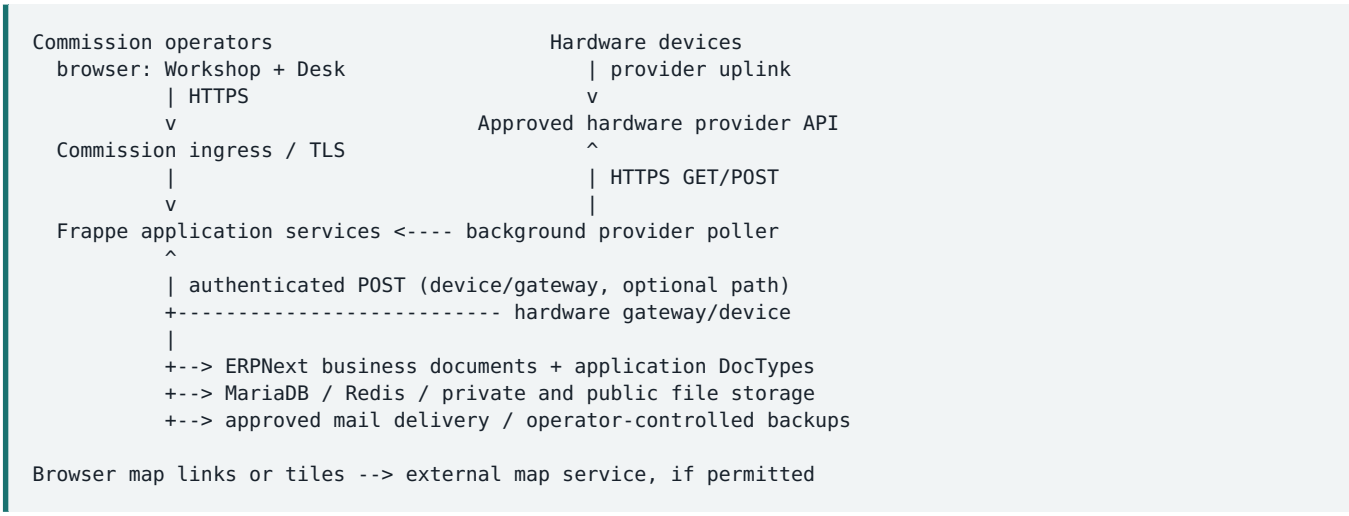
1. Scope and Design Drivers

Describe the whole application and its intended Docker deployment: logical components, data ownership, trust boundaries, deployment services, processing flows, failure handling and operating procedures. The repository supplies an application image Dockerfile and a Compose **override**, not a standalone production platform. No final Commission Compose stack, registry digest, firewall rules or hosting evidence was supplied.

Customer: Addis Ababa Police Commission. Hosting: planned on-premise on the Commission's server. Planning population: more than 700 vehicles and more than 500 users; peak concurrency and telemetry rate are not yet measured. Technical contact: Natnael Tilaye (natnaeltilaye30@gmail.com).

The principal drivers are coherent vehicle/workshop records, reuse of ERPNext transactions, a usable workshop interface, site-specific installation convergence, authenticated hardware integration and repeatable container delivery. Production assurance additionally requires consistent authorization, controlled data handling, backup/restore and independently verified infrastructure.

2. System Context



No inbound hardware provider access to MariaDB, Redis or worker ports is required. The public ingress terminates and routes application traffic; protected backing services remain on private networks. The actual deployment owner must prove this isolation with the final Compose/network configuration.

2.1 External Dependencies

Dependency	Purpose	Production decision
Hardware provider API	Latest GPS/engine/fuel snapshots	Endpoint, account scope, SLA, units and permitted egress
Device/gateway network	Optional authenticated push	Credential distribution, TLS, destination retry and buffering
DNS and time service	Host resolution and timestamp consistency	Approved resolvers and synchronized clock source
Map service	GPS link and frontend mapping where used	Information disclosure, internet dependency and approved alternative
Email	Scheduled report delivery and configured notifications	Recipient control, mail account and transport security
Registry/source/dependency services	Controlled build and release delivery	Trusted sources, immutable versions, scans and artifact retention
Backup destination	Off-instance recovery	Encryption, access, location, retention and restore evidence

The provider credentials must not be shared with workshop browsers. Frontend test actions call the backend, which makes the provider request. A user who can test a provider may see the returned fleet data; provider account scope must therefore be minimal.

3. Logical Architecture

Layer	Components	Responsibility
Presentation	Vue 3, TypeScript, Pinia, Vue Router, Tailwind; Frappe Desk	Workshop workflows, data views, forms and native administration
Web bootstrap	`www/workshop.py`, route rules and built Vite manifest	CSRF/user bootstrap and current hashed asset resolution
Transport	Frappe method/resource APIs, session or API token	Request parsing, authentication and framework responses
Application services	`api/*`, RO controller, Vehicle overrides	Fleet, workshop, finance, inspections, reports and integration orchestration
Hardware adapter	`api/hardware.py`, `api/telemetry.py`	External fetch, parse, IMEI resolution and normalized telemetry storage
Background processing	`polling.py`, `tasks.py`, Frappe scheduler/RQ	Conditional polling, scheduled reports, daily costing
Persistence	ERPNext records, shipped application DocTypes, setup-created telemetry DocTypes, files	Transactional business data and sensor histories
Platform	Frappe/ERPNext, MariaDB, Redis, Socket.IO, web/queue processes	ORM, metadata, jobs, identity, cache and realtime transport

The SPA is not an independent backend: it uses the same site/session and API origin. Generic resource calls follow framework DocType permissions, while application method endpoints need explicit authorization. Some legacy endpoints bypass DocType permissions; the Security document identifies this gap rather than attributing uniform enforcement to the framework.

4. Deployment Architecture

```
Internet or approved Commission client/device network
                    |
                HTTPS :443
                    |
      [TLS ingress / reverse proxy]  operator supplied
                    |
             [frontend: nginx]
               |           |
       [backend: web]   [websocket]
               |           |
          private application network
          |         |          |
      [MariaDB] [Redis cache] [Redis queue]
          ^                    ^
          |                    |
    [queue-short] [queue-long] [scheduler]

Persistent: database data, site configuration/files and operational logs
One-time/configuration service: configurator
```

The service names above match the supplied override. Ports, replicas, healthchecks, resource limits and network names are inherited from the deployment's base Compose files; they are not fixed by this application override. Do not assume the development bench ports or process counts are the production values.

4.1 Docker Package Contract

`docker/Dockerfile` defaults to `frappe/erpnext:v15.79.0`, copies this application from a **bench-root build context**, installs its Python package into the image virtual environment, registers `car_repair_management` in `sites/apps.txt`, builds the Vue bundle and runs the bench asset build. It then checks the built asset directory. The hardcoded Node path targets v18.18.2; compatibility with the selected base image must be demonstrated during release build.

`docker/compose.car-repair-management.yaml` selects `car-repair-management:v15.79.0` for backend, configurator, frontend, websocket, queue-short, queue-long and scheduler. The identical application version must run in all these services. A mismatch can serve missing frontend chunks, execute outdated worker code or import the wrong schema.

The supplied Dockerfile does not pin the upstream image by digest or generate an SBOM/signature. It copies the app directory, so release build contexts must exclude credentials, site dumps, local dependency directories and unnecessary Git history. A clean controlled source export, reviewed `.dockerignore`/build context, locked dependencies and registry scanning are release requirements, not controls proven by the current Dockerfile.

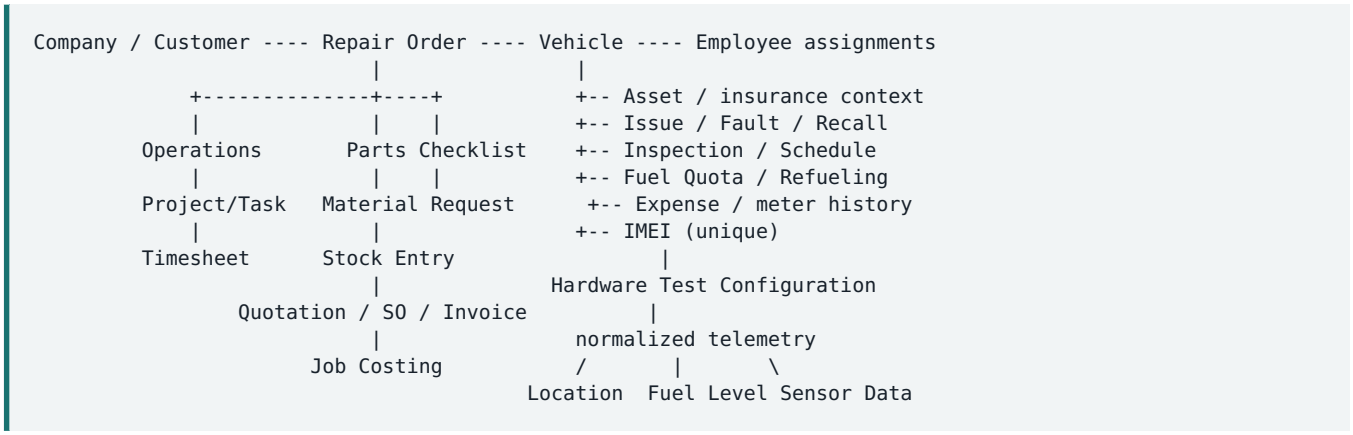
4.2 Persistence and Assets

Storage	Contents	Required protection
MariaDB volume	Business records, permissions, telemetry and metadata	Private connectivity, restricted DB account, consistent backups
Sites volume	Site configuration, encryption key, file attachments and app list	Restricted access and recoverable encrypted backup
Public files	Deliberately public attachments/assets	Do not put operational personal/GPS documents here
Private files	Access-controlled documents and receipts	Test download authorization; filesystem copies need encryption

Storage	Contents	Required protection
Assets mounts	Compiled frontend and linked application assets	Consistent version across frontend/ backend; disposable only when positively identified
Redis	Cache, queue state and polling lease	Private only, access-controlled, recovery implications reviewed
Logs	Service errors and operator evidence	Redaction, retention, protected central collection

Existing mounted assets can mask new image assets. Inspect container mounts and verify direct hashed-asset responses during an upgrade. Never run a blanket volume prune or delete database/sites volumes to solve a blank workshop screen. Only explicitly identified disposable assets mounts may be recreated under an approved maintenance plan.

5. Data Design



This is a conceptual relationship diagram, not a declaration that every association has a database-level foreign-key constraint. Frappe Link fields and application validation enforce many relationships. The exact shipped field definitions, child tables, links, required flags and permission rows are in the generated TDD appendix.

Telemetry DocTypes (`Vehicle Location`, `Vehicle Fuel Level`, `Vehicle Sensor Data`) are created by installation code, then enriched with traceability fields. ERPNext Vehicle and other standard documents receive custom fields. Therefore copying shipped DocType JSON alone is not sufficient installation; site setup/migration is part of the application contract.

5.1 Data Ownership

- The Commission owns operational fleet, personnel associations, repair and financial records and approves their access/retention.
- Provider observations are externally sourced measurements; provenance, units and timestamp trust must be recorded in the integration agreement.
- ERPNext remains the transaction system for stock and finance. Dashboard totals and replacement scores are views/derived values, not independent financial approval evidence.
- The service owner approves backup and report copies. A public repository must never become a store of live fleet traces or production configuration.

6. Processing Designs

6.1 Workshop Work Order

1. An authorized operator creates a draft RO referencing Vehicle and Company/Customer.
2. Validation checks order party, parts flags and SLA; before-save recomputes relevant cost fields.
3. Submit persists Scheduled status and applies a selected service template. Project/tasks are created by Start Work as needed, not by submission alone.
4. Task progress and commercial/stock/time documents refer back to the RO. Event hooks update costs/status.
5. A Ready for Handover transition checks QC task completion. Checklist/inspection data provides handover context.
6. Closure uses the currently implemented linked-invoice status guard; the Commission must decide whether to strengthen it to enforce actual settlement and mandatory invoice presence.

6.2 Provider Configuration and Mapping

1. Integration manager enters a draft endpoint/key and optional request/response settings.
2. Backend Test validates the URL, checks resolved addresses, fetches with bounds, parses and redacts preview output. No configuration or telemetry is created by Test.
3. Save persists the configuration; the table lists saved rows. Activate serializes active selection so the enabled default is unique.
4. A vehicle uses a unique IMEI and either an explicit enabled API or the active API. Vehicle read/write permission and integration-manager role are required for connection-changing/manual actions.
5. Test GPS & Fuel previews the saved mapping. Sync Now ingests matching data; Find Devices assists selection but does not map by plate/name.

6.3 Automatic Polling

```
configuration/vehicle committed change OR minute scheduler watchdog
    -> eligible mapped vehicles exist?
    -> acquire site Redis lease (180 seconds)
    -> enqueue bounded long-queue worker (120-second job timeout)
    -> work window up to 45 seconds, one-second checks
        -> group vehicles by selected provider
        -> find due vehicles using last condition + configured interval
        -> fetch provider once for a due group
        -> per due vehicle: parse, hash, optionally ingest, store status
        -> commit independently per provider
    -> release owned lease and enqueue successor while eligible
```

Running = engine on and speed greater than zero; Idle = engine on without movement or explicit idle tracker status; Off = explicit off. Unknown/error uses the shortest configured interval. The next schedule uses the last fetched condition, not an unobserved state change. Request-completion time is recorded for due vehicles in the group. Repeated stable snapshots skip ingestion; changing a human-readable elapsed-stop timer alone is not a new sample.

The lease limits duplicate workers under ordinary operation, but it is not a formal exactly-once guarantee under lease expiry, process timeout or infrastructure failure. A busy long queue delays subsequent windows. Allocate and test adequate managed long-worker capacity and monitor lateness. One-second configured intervals across many providers must not be promised without load/provider-rate testing.

6.4 Push and Dashboard Refresh

A device/gateway sends an authenticated provider-shaped payload to the central telemetry method. The server validates values, resolves the mapped enabled vehicle and stores location/fuel/sensor rows with a batch ID. Dashboard live fields resist older timestamps. Push ingestion does not share the poller's stable-snapshot deduplication guarantee.

Visible vehicle detail pages reload stored dashboard data every five seconds. This is separate from provider polling and does not multiply hardware calls per browser. A realtime telemetry event is also published by the poller after commit; the current visible-page refresh does not depend on it. Measurement timestamp, provider server timestamp and last request time serve different purposes.

7. Trust Boundaries and Security Responsibilities

Boundary	Threat	Primary control / limitation
Client/device to ingress	Credential theft, request abuse	HTTPS, token/session auth, ingress rate/body limits; actual rules pending
Authenticated user to application	Unauthorized records/actions	Framework permissions plus explicit API checks; legacy gaps remain
Backend to provider	SSRF, credential forwarding, poisoned response	Public-address check, no redirects, timeouts/size/parser bounds; egress allowlist still required
App to storage	Broad DB/file access and data leakage	Private services and least-privilege operations; host/volume encryption operator responsibility
Operator to deployment	Wrong image, secrets in build, destructive migration	Reviewed immutable release, backups, scoped commands and controlled access
App to reports/maps/email	Unapproved disclosure	Approved recipients/destinations, export policy and data classification

Passwords use Frappe Password fields; recovery depends on protecting the site encryption key. This is not evidence of whole-database encryption. Redis/site names provide operational site scoping, not a substitute for tenant-boundary authorization tests or separate infrastructure for higher-assurance isolation.

8. Deployment, Upgrade and Rollback

The release engineer executes these under a change record after staging verification. `SITE`, paths, image tags and Compose flags below are illustrative operator variables, not validated Commission values.

```
# From a clean, reviewed bench-root build context
docker build --build-arg FRAPPE_VERSION=v15.79.0 \
  -f apps/car_repair_management/docker/Dockerfile \
  -t car-repair-management:APPROVED_RELEASE .

# In the controlled deployment directory, using the complete approved file list
docker compose -f compose.yaml -f APPROVED_OVERRIDES.yaml config --quiet
docker compose -f compose.yaml -f APPROVED_OVERRIDES.yaml up -d
```

Use the **same full Compose flags/project/env files** for every subsequent command; the following assumes the operator has configured that full stack selection. Do not accidentally operate on a different project using an incomplete command.

```
docker compose exec backend bench --site "$SITE" list-apps
# New site only, after ERPNext setup:
docker compose exec backend bench --site "$SITE" install-app car_repair_management
docker compose exec backend bench --site "$SITE" migrate
docker compose exec backend bench --site "$SITE" clear-cache
docker compose exec backend bench --site "$SITE" execute car_repair_management.api.health.audit_site
docker compose exec backend bench --site "$SITE" enable-scheduler
docker compose exec backend bench doctor
docker compose ps
```

Confirm the chosen image tag is set in all seven override services. Back up an existing site **before** migrations, including files and protected site configuration/key. Drain or stop appropriate writes/polling during migration. Recreate web, workers, scheduler and frontend through the managed container deployment; host `bench restart`/Supervisor process-kill recipes are not the Docker update procedure.

Verify `/workshop`, Desk, direct current assets, protected API access, saved configuration listing, synthetic IMEI test and recurring polling. Repeat migration/audit for every installed site. Review and remove seeded demo data using a controlled review, not a blanket production delete.

Rollback may require restoring the pre-upgrade database, files and encryption configuration with the previous image. Merely changing the image tag cannot undo a schema/data migration. Test restore in an isolated environment, reconcile writes made after the backup, and obtain business approval before resuming traffic.

9. Capacity, Observability and Recovery

Capacity planning must include application sessions, dashboard five-second refreshes, concurrent report queries, provider group frequency, sensors per sample, file storage and telemetry retention. A changing sample can create a location, optional fuel row and multiple sensor rows; storing every sensor value has substantially greater growth than counting HTTP requests alone.

Monitor ingress errors/latency, login failures, provider status/latency, last request vs next due, measurement age, queue waiting time, worker restarts, Redis/DB availability, storage growth, backup age and restore success. Define escalation thresholds after load testing. Configuration last-run fields primarily reflect manual Run; automatic polling status is stored on Vehicle.

Provider failure is recorded per vehicle and retried at the shortest configured interval; no exponential circuit breaker or automatic alternative-provider selection is claimed. A stale provider snapshot may yield successful requests without fresh GPS. A database failure cannot be handled as a trustworthy stored dashboard update; backing services require managed recovery.

Availability design, replicas, failover, RPO/RTO, encrypted off-instance backups and disaster recovery topology remain operator decisions pending approval. The supplied override alone does not implement high availability.

10. Design Decisions and Verification

Decision	Reason	Consequence / verification
Reuse Frappe/ERPNext	Existing document, stock and finance model	Version compatibility and permission review are mandatory
Vue SPA plus Desk	Ergonomic workflows with native advanced editing	Both entry points and direct APIs must be tested
Backend provider adapter	Keep provider key out of browser, share parser	SSRF/egress and full-response visibility need controls
IMEI mapping	Stable hardware identity, independent of display name	Device replacement requires controlled reassignment

Decision	Reason	Consequence / verification
Bounded long-queue polling	Sub-minute targets with existing worker stack	Queue capacity and recovery/lease tests required
Per-site setup convergence	Avoid manual-field drift across installations	Every site migrates; audit is not security assurance
Baked assets on all Frappe services	Prevent frontend/backend asset mismatch	Mounted volumes and immutable image selection need verification

Source references: `hooks.py`, `install.py`, `polling.py`, `tasks.py`, `api/hardware.py`, `api/telemetry.py`, Vehicle/RO controllers, `www/workshop.py`, frontend router/client/pages, and the three `docker/` files at the stated baseline. See TDD inventory and `verification.md` for reproducible evidence. Architect sign-off, production topology review and deployment rehearsal record: TO BE CONFIRMED.